

Question 1: Can you explain your CommerceIQ project end-to-end?

Answer:

CommerceIQ is an end-to-end e-commerce growth and experimentation analytics platform built to simulate how a real analytics team would work with transactional, customer, marketing, funnel, and experiment data.

The core transactional dataset is the Olist Brazilian e-commerce dataset. I first profiled and validated the raw data using Python, then loaded it into PostgreSQL.

From there, I used dbt to create a layered transformation pipeline:

Raw data → Staging models → Intermediate models → Fact/Dimension models → Business marts

The core warehouse follows a star-schema approach with models such as `fct_orders`, `fct_order_items`, `dim_customers`, `dim_products`, `dim_sellers`, and `dim_date`.

On top of that, I built business marts for:

- customer segmentation and cohort retention,
- product and seller performance,
- delivery and customer experience,
- funnel and marketing analytics,
- and A/B experimentation.

Because the original Olist dataset does not contain digital clickstream or experiment data, I generated clearly labeled synthetic datasets for website sessions, marketing spend, purchase value, and checkout experimentation.

For experimentation, I created a randomized control-treatment setup, checked for sample ratio mismatch, ran a two-proportion z-test, calculated uplift and confidence intervals, and estimated business impact.

Finally, I connected the analytical layer to Power BI and built a six-page dashboard covering executive KPIs, customer retention, products and sellers, operations, funnel and marketing, and experimentation.

The project demonstrates the complete analytics lifecycle:

**raw data → data modeling → analytics engineering → statistical analysis
→ BI → business decision-making.**

Question 2: Why did you use dbt in this project instead of writing all transformations directly in SQL or Power BI?

Answer:

I used dbt because I wanted the transformation layer to be modular, testable, reusable, and easier to maintain.

Instead of writing one large SQL query, I separated the logic into layers:

Staging → Intermediate → Fact/Dimension models → Business marts

This gave me several advantages:

- **Modularity:** each model performs one clear transformation.
- **Reusability:** common logic can be reused across multiple marts.
- **Testing:** I could add uniqueness, not-null, relationship, and custom business-rule tests.
- **Documentation:** model purpose and dependencies are easier to understand.
- **Lineage:** dbt makes it clear how raw data flows into final analytical outputs.
- **Maintainability:** if business logic changes, I can update one upstream model instead of editing many dashboard queries.

I also kept most business logic outside Power BI because I wanted Power BI to focus mainly on visualization and light semantic calculations, while PostgreSQL and dbt handled the heavier transformation work.

That makes the architecture closer to an industry analytics workflow and reduces the risk of duplicating logic across reports.

Question 3: Why did you choose a star schema for this project?

Answer:

I used a star schema because it makes analytical querying and Power BI reporting simpler, faster, and easier to maintain.

The main fact tables are:

- fct_orders
- fct_order_items

And the main dimensions are:

- dim_customers
- dim_products
- dim_sellers
- dim_date

This structure separates measurable business events from descriptive attributes.

For example, fct_orders contains order-level metrics such as order value, delivery performance, payment information, and review score, while dimensions provide descriptive context like customer, product, seller, and date attributes.

The main benefits were:

- simpler joins for BI,
- consistent metric definitions,
- easier slicing and filtering,
- better Power BI relationship management,
- reduced duplication of descriptive data,
- and better scalability when adding new analytical marts.

I also used a separate fct_order_items table because order-level and item-level analysis have different grains. Keeping them separate avoids incorrect aggregation and double counting.

Question 4: How did you ensure data quality and correctness in this project?

Answer:

I treated data quality as part of the analytics pipeline rather than checking it only at the dashboard stage.

I used a combination of source profiling, dbt tests, and custom reconciliation checks.

The main validations included:

- **Uniqueness tests** for primary keys such as order IDs.
- **Not-null tests** for critical fields.
- **Relationship tests** between fact and dimension models.
- **Accepted-value tests** for controlled categorical fields.
- **Business-rule tests** for cases where standard dbt tests were not enough.
- **Reconciliation tests** to verify that aggregated marts matched their underlying source models.

For example, I created custom tests to validate:

- funnel-stage logic,
- monthly funnel reconciliation,
- marketing ROAS calculations,
- experiment-result reconciliation,
- cohort-retention validity,
- and seller-concentration calculations.

I also manually validated important business assumptions. One example was review timing: instead of directly comparing timestamps, I validated the business logic using the appropriate delivery and review dates, which materially changed the late-delivery analysis.

The final dbt build completed with:

480 passes, 1 expected warning, 0 errors.

The only warning was related to known missing product metadata in the source dataset.

This approach helped ensure that the dashboard metrics were not only visually correct, but also traceable and analytically reliable.

Question 5: What were the most important business insights you found from the project?

Answer:

The project produced several important business insights across customer behavior, operations, marketplace performance, and growth.

The most important findings were:

- **Repeat customers were only about 3% of the customer base**, which showed that repeat purchasing was relatively low.
- However, repeat customers generated roughly **1.9x higher lifetime GMV proxy** than one-time customers, making retention strategically important.

- **Late deliveries were strongly associated with poor customer experience.** Late orders had about **2.7× higher low-rating risk** than on-time orders.
- Product GMV was concentrated in a limited number of categories, with the **top 10 categories contributing roughly 62% of GMV.**
- Seller GMV was comparatively diversified, so the marketplace was **not heavily dependent on a few sellers.**
- In the synthetic digital funnel, the biggest drop-off occurred between **Product View and Add to Cart**, making that stage the main optimization opportunity.
- In the synthetic marketing layer, Email had the strongest simulated efficiency, with lower cost per purchase and higher ROAS than the other modeled channels.
- In the randomized checkout experiment, treatment conversion was higher than control by about **3.9 percentage points**, and the result was statistically significant.

The key value of the project was that these insights were not isolated metrics. They were connected to clear business actions around retention, delivery improvement, category focus, funnel optimization, and experimentation.

Question 6: Why did you use January 2017 to August 2018 as the main analysis period?

Answer:

The raw Olist dataset contains orders from late 2016 through 2018, but the first and last parts of the dataset are incomplete.

If I used those partial months in trend analysis, metrics such as monthly orders, GMV, and customer activity could appear artificially low simply because the dataset does not contain the full month.

To avoid that distortion, I defined a reliable analysis window:

January 2017 to August 2018

I implemented this logic in `dim_date` using an `is_reliable_analysis_period` flag and used that flag in important dashboard measures.

This helped ensure that month-over-month comparisons were made only across complete periods.

I still retained the full source data for validation and other analysis, but I used the reliable window for executive trend KPIs.

The main principle was:

Do not treat incomplete boundary periods as directly comparable to complete months.

Question 7: How did you define and calculate customer retention and repeat customers in this project?

Answer:

I used `customer_unique_id` as the persistent customer identifier because the Olist dataset can contain different `customer_id` values for the same real customer across different orders.

I classified a customer as a **repeat customer** if the same `customer_unique_id` appeared in more than one delivered order within the reliable analysis period.

From there, I calculated:

- total unique customers,
- repeat customers,
- repeat customer rate,
- customer lifetime GMV proxy,
- cohort retention,
- and RFM-based customer segments.

For cohort retention, I assigned each customer to a cohort based on their **first purchase month** and then tracked whether they returned in later months.

This allowed me to answer two different questions:

- **Repeat rate:** What percentage of customers purchased more than once?
- **Cohort retention:** How well do customers from each acquisition cohort return over time?

That distinction was important because repeat rate is a simple overall metric, while cohort retention shows retention behavior across time.

Question 8: How did you perform customer segmentation in this project?

Answer:

I used **RFM analysis** to segment customers based on three dimensions:

- **Recency** — how recently the customer purchased
- **Frequency** — how many orders the customer placed
- **Monetary value** — how much GMV the customer generated

I first calculated customer-level metrics in dbt, then created RFM scores using percentile-based scoring so that the segments were more robust to skewed distributions.

This was especially important because the dataset has a very large number of one-time buyers, so a simple frequency-based segmentation would not have been very informative.

Using the RFM scores, I created business-friendly segments such as:

- Champions
- Loyal Repeat
- High-Value Recent
- At-Risk High Value
- Promising
- Regular

- Hibernating

The goal was not just to rank customers, but to create segments that could support actions such as retention campaigns, win-back strategies, and differentiated engagement.

The key advantage of RFM here is that it converts raw transaction history into actionable customer groups.

Question 9: Why do you call the customer metric an LTV proxy instead of true Customer Lifetime Value?

Answer:

I call it an **LTV proxy** because the dataset allows me to estimate the total GMV generated by a customer across their observed purchase history, but it does not contain all the inputs required for true financial LTV.

True Customer Lifetime Value should ideally consider factors such as:

- gross margin,
- acquisition cost,
- servicing cost,
- refunds or returns,
- contribution margin,
- and expected future customer behavior.

In this project, I calculated customer lifetime value using observed **lifetime GMV**, so it measures customer value from a transaction-volume perspective rather than profitability.

That is why I intentionally label it:

LTV Proxy

instead of claiming it is true profit-based LTV.

This distinction is important because analytics metrics should reflect exactly what the underlying data can support.

Question 10: How did you analyze the impact of late delivery on customer experience?

Answer:

I compared delivered orders based on whether they arrived **on time or late**, and then measured customer review outcomes for each group.

The main metrics I compared were:

- average review score,
- low-rating rate,
- and reviewed-order counts.

The results showed:

- **On-time orders:** average review score of about **4.29**, with a low-rating rate of about **9.2%**
- **Late orders:** average review score of about **3.53**, with a low-rating rate of about **25.1%**

So late deliveries had roughly:

2.7× higher low-rating risk

This made delivery performance one of the strongest operational signals associated with customer dissatisfaction in the project.

However, I would not call this causal because the data is observational. Other factors such as seller behavior, product type, distance, or order complexity may also influence reviews.

So the correct conclusion is:

Late delivery is strongly associated with poorer customer experience, but the analysis does not by itself prove causation.

Question 11: How did you analyze seller concentration, and what did you conclude?

Answer:

I analyzed seller concentration to understand whether marketplace GMV was dominated by a small number of sellers or distributed across many sellers.

I calculated metrics such as:

- Top 1 seller GMV share
- Top 5 seller GMV share
- Top 10 seller GMV share
- Top 20 seller GMV share
- Seller HHI, or Herfindahl-Hirschman Index

The approximate GMV shares were:

- Top 1 seller: **~1.7%**
- Top 5 sellers: **~7.7%**
- Top 10 sellers: **~13.3%**
- Top 20 sellers: **~21.3%**

These values indicate that seller GMV is relatively diversified rather than being concentrated among a few dominant sellers.

That is useful from a marketplace-risk perspective because the business is less dependent on a very small number of sellers for its GMV.

I also used seller-level operational marts to separately identify sellers with higher late-delivery risk, so concentration and seller quality were treated as different analytical questions.

Question 12: Why did you create synthetic funnel, marketing, and experiment data instead of using only the Olist dataset?

Answer:

The Olist dataset is strong for transactional, customer, seller, product, payment, review, and delivery analysis, but it does not contain important digital-product data such as:

- website sessions,

- funnel events,
- acquisition channels,
- marketing spend,
- campaign performance,
- or controlled experiment assignments.

Since I wanted the project to represent a more complete modern e-commerce analytics workflow, I created clearly labeled synthetic datasets for those missing areas.

I used synthetic data to add:

- website sessions and funnel events,
- marketing spend by channel,
- website purchase value,
- and randomized A/B experiment data.

The important point is that I did **not** mix synthetic data with real transactional data without disclosure. In the dashboard and documentation, I clearly labeled the digital and experimentation layers as synthetic.

This allowed me to demonstrate additional industry-relevant skills such as:

- funnel analysis,
- channel efficiency,
- ROAS,
- cost per purchase,
- experiment design,
- statistical significance,
- confidence intervals,
- and SRM checks.

So the synthetic layer was not added to fabricate business results; it was added to extend the project into analytics areas that the original dataset simply does not support.

Question 13: How did you build and analyze the conversion funnel in this project?

Answer:

I created the funnel using synthetic website session and event data because the original Olist dataset does not contain clickstream information.

The funnel stages were:

Session Start → Product View → Add to Cart → Checkout → Purchase

I first converted event-level data into a **session-level funnel model**, where each session had flags indicating whether the user reached each stage.

Then I calculated:

- number of sessions reaching each stage,
- stage-to-stage conversion rates,
- drop-off counts,

- overall session-to-purchase conversion,
- and conversion by channel and device.

The approximate funnel performance was:

- Session → Product View: ~**77.9%**
- Product View → Add to Cart: ~**24.7%**
- Add to Cart → Checkout: ~**55.0%**
- Checkout → Purchase: ~**61.8%**
- Overall Session → Purchase: ~**6.5%**

The biggest drop-off occurred between:

Product View → Add to Cart

Around three-quarters of product-view sessions did not add a product to cart.

So if this were a real business, I would investigate areas such as product-page quality, pricing, trust signals, shipping information, and CTA effectiveness at that stage.

Because the funnel data is synthetic, I treat this as an analytical demonstration rather than a real business diagnosis.

Question 14: How did you evaluate marketing channel performance in the project?

Answer:

I evaluated marketing channels using the synthetic marketing and website-purchase datasets.

The main channels were:

- Email
- Social
- Paid Search

For each channel, I calculated:

- impressions,
- clicks,
- CTR,
- CPC,
- CPM,
- website sessions,
- purchases,
- cost per session,
- cost per purchase,
- and ROAS.

The approximate results showed:

- **Email** had the lowest cost per purchase and the highest ROAS.
- **Social** performed in the middle.
- **Paid Search** had the highest cost per purchase and the lowest ROAS among the three.

For example, approximate cost per purchase was:

- Email: ~**R\$64.86**
- Social: ~**R\$87.02**
- Paid Search: ~**R\$126.49**

And approximate ROAS was:

- Email: ~**2.30x**
- Social: ~**1.73x**
- Paid Search: ~**1.17x**

I deliberately called the efficiency metric **cost per purchase**, not CAC, because the data does not prove that every purchaser is a new customer.

Also, since this marketing layer is synthetic, these results are used to demonstrate the analytical workflow rather than make claims about a real marketing program.

Question 15: How did you design the A/B experiment in this project?

Answer:

I designed the experiment to test whether an improved checkout experience increases the **checkout-to-purchase conversion rate**.

The experiment had two groups:

- **Control** — existing checkout experience
- **Treatment** — improved checkout experience

The **randomization unit was visitor_id**, so the same visitor could not be placed in both groups.

Only visitors who reached checkout were eligible for the experiment, which gave an experiment population of about **11,706 visitors**.

The allocation was approximately 50/50:

- Control: **5,865 visitors**
- Treatment: **5,841 visitors**

Before analyzing conversion, I also considered:

- significance level: **5%**
- statistical power: **80%**
- minimum detectable effect
- required sample size
- sample ratio mismatch
- and avoiding repeated peeking at the results

The primary metric was:

Checkout-to-Purchase Conversion Rate

The observed conversion rates were approximately:

- Control: **60.84%**
- Treatment: **64.73%**

So the treatment produced an absolute uplift of about:

+3.90 percentage points

The key design principle was that assignment happened randomly at the visitor level, which allowed the treatment effect to be interpreted causally within the synthetic experiment.

Question 16: Why did you perform a Sample Ratio Mismatch (SRM) check before evaluating the experiment?

Answer:

I performed an SRM check to verify that the observed number of users in the control and treatment groups was consistent with the intended randomization.

The experiment was designed for approximately a **50/50 split**.

Observed counts were:

- Control: **5,865**
- Treatment: **5,841**

I used a chi-square test to compare the observed allocation with the expected allocation.

The SRM p-value was approximately:

0.8245

Since this is much higher than 0.05, there was no evidence of a sample ratio mismatch.

This matters because a strong imbalance between control and treatment can indicate problems such as:

- broken randomization,
- tracking issues,
- eligibility inconsistencies,
- assignment bugs,
- or missing data.

If SRM had been detected, I would not trust the experiment result until the cause was investigated.

So SRM acts as an **experiment validity check before analyzing treatment impact**.

Question 17: Which statistical test did you use to compare control and treatment conversion rates, and why?

Answer:

I used a **two-proportion z-test** because the experiment compares conversion rates between two independent groups:

- Control
- Treatment

The outcome is binary:

- Purchase = 1

- No Purchase = 0

So the metric of interest is a proportion in each group.

The null hypothesis was:

H_0 : Control conversion rate = Treatment conversion rate

The alternative hypothesis was:

H_1 : Control conversion rate $\neq$ Treatment conversion rate

The observed conversion rates were approximately:

- Control: **60.84%**
- Treatment: **64.73%**

The test produced:

- **$z \approx 4.36$**
- **$p \approx 0.000013$**

Since the p-value is much smaller than the significance level of **0.05**, I rejected the null hypothesis and concluded that the observed difference was statistically significant.

The two-proportion z-test was appropriate here because I was comparing two large independent samples with binary outcomes.

Question 18: What is the difference between statistical significance and business significance in your A/B test?

Answer:

Statistical significance tells me whether the observed difference between control and treatment is likely to be real rather than random noise.

In my experiment:

- Control conversion rate: **~60.84%**
- Treatment conversion rate: **~64.73%**
- Absolute uplift: **~3.90 percentage points**
- p-value: **~0.000013**
- 95% confidence interval for uplift: **~2.15 to 5.65 percentage points**

Because the p-value is far below 0.05 and the confidence interval does not include zero, the result is statistically significant.

But statistical significance alone is not enough.

Business significance asks:

Is the size of the improvement large enough to matter commercially?

To evaluate that, I translated the uplift into projected impact.

At **100,000 checkout visitors**, the observed effect corresponds to roughly:

3,897 additional purchases

with an approximate confidence interval of:

2,147 to 5,646 incremental purchases

So in this case, the treatment was both:

- **statistically significant**
- and potentially **business-significant**

The key principle is:

A very small effect can be statistically significant with a large sample, so I always evaluate both the statistical evidence and the practical business impact.

Question 19: How did you calculate and interpret the 95% confidence interval for the A/B test uplift?

Answer:

I calculated a **95% confidence interval** around the difference in conversion rates between treatment and control.

The observed absolute uplift was approximately:

+3.90 percentage points

The 95% confidence interval was approximately:

+2.15 percentage points to +5.65 percentage points

I interpret this as the range of plausible values for the true treatment effect under the assumptions of the test.

The important part is that the entire interval is **above zero**.

That means the data supports a positive treatment effect, which is consistent with the very small p-value.

The confidence interval is especially useful because it gives more information than the p-value alone.

The p-value tells me whether the result is statistically significant, while the confidence interval tells me:

- the likely range of the uplift,
- the uncertainty around the estimate,
- and whether the effect is large enough to be meaningful from a business perspective.

So for decision-making, I would always look at both the **p-value and the confidence interval**, not only statistical significance.

Question 20: Why did you use Power BI on top of dbt marts instead of connecting directly to raw tables?

Answer:

I connected Power BI mainly to curated dbt models rather than raw source tables because I wanted the dashboard layer to stay simple, consistent, and performant.

The heavy transformation logic was handled upstream in PostgreSQL and dbt.

For example, dbt already prepared marts for:

- monthly KPIs,
- customer segments,
- cohort retention,
- seller performance,
- delivery experience,
- funnel performance,
- marketing efficiency,
- and experiment results.

This gave me several advantages:

- **Consistent metrics:** business logic is defined once upstream.
- **Better performance:** Power BI reads smaller, pre-aggregated tables instead of recalculating everything from raw data.
- **Less DAX complexity:** DAX is mainly used for reporting measures and presentation logic.
- **Easier maintenance:** if business logic changes, I update the dbt model rather than multiple dashboard visuals.
- **Better governance:** the same analytical mart can support multiple reports without duplicating logic.

I still used the core star-schema tables where interactive slicing was needed, but for specialized analytical pages I used purpose-built aggregate marts.

The overall principle was:

Keep transformation and business logic upstream, and use Power BI primarily for semantic modeling, interaction, and visualization.

Question 21: What calculations did you keep in DAX, and what calculations did you keep in dbt/SQL?

Answer:

I used a simple rule:

Heavy transformation and reusable business logic stayed in dbt/SQL, while report-level calculations stayed in DAX.

In dbt/SQL, I handled logic such as:

- customer cohorts,

- RFM scoring and segmentation,
- seller concentration,
- delivery-performance metrics,
- funnel stage calculations,
- marketing performance marts,
- experiment summary tables.

These calculations are reusable and should remain consistent across reports.

In DAX, I created measures mainly for dashboard interaction and presentation, such as:

- Total GMV
- Reliable Delivered Orders
- Average Order Value
- Active Customers
- Late Delivery Rate
- Repeat Customer Rate
- Average LTV Proxy
- experiment display measures such as significance status and confidence-interval display.

This separation helped me avoid putting complex transformation logic inside Power BI.

The design principle was:

SQL/dbt defines the business logic; DAX handles dynamic report calculations and presentation.

Question 22: How did you handle different data grains and prevent double counting in the project?

Answer:

One important challenge was that different Olist tables exist at different grains.

For example:

- orders → one row per order
- order_items → multiple rows per order
- payments → potentially multiple payment records per order
- reviews → potentially multiple review records per order

If I joined all of these raw tables directly, an order with multiple items and multiple payments could be duplicated, which would inflate metrics such as GMV, freight, or order counts.

To prevent this, I first defined the grain of every model.

For example:

- fct_orders → **one row per order**
- fct_order_items → **one row per order item**
- dim_customers → **one row per unique customer**

Before building fct_orders, I aggregated one-to-many tables to the order level using intermediate models such as:

- int_order_payments
- int_order_items_summary
- int_latest_order_review

Then I joined those order-level outputs to the order table.

For item-level product and seller analysis, I used fct_order_items instead of the order-level fact.

The main principle was:

Always define the grain before joining tables and aggregate one-to-many relationships to the required grain before calculating business metrics.

This helped prevent duplicated rows and incorrect KPIs.

Question 23: What was the most challenging part of this project, and how did you solve it?

Answer:

One of the biggest challenges was maintaining **metric correctness across multiple data grains and analytical layers**.

The project combined:

- order-level data,
- item-level data,
- customer-level metrics,
- seller-level metrics,
- funnel sessions,
- marketing data,
- and experiment results.

If these were joined or aggregated incorrectly, metrics such as GMV, order counts, payment totals, or conversion rates could become inflated.

I solved this by:

- defining the grain of each model before writing transformations,
- using intermediate dbt models to aggregate one-to-many tables before joining them,
- separating fct_orders and fct_order_items,
- creating purpose-specific marts for customer, seller, funnel, marketing, and experiment analysis,
- and adding reconciliation tests to verify that final aggregates matched their source models.

Another challenge was distinguishing observational findings from causal conclusions.

For example, late delivery was strongly associated with poor ratings, but I did not claim causality because that analysis was observational.

In contrast, the synthetic checkout experiment used randomized assignment, so I could interpret the treatment effect causally within the simulation.

The main lesson was that building the dashboard was not the hardest part; the harder part was ensuring that the numbers were logically valid and analytically defensible.

Question 24: If this project had to be moved to production, what would you improve?

Answer:

The current project is designed as a portfolio-scale analytics platform, so for production I would strengthen automation, scalability, and monitoring.

The main improvements would be:

- Move the analytical warehouse from local PostgreSQL to a scalable cloud warehouse such as **BigQuery, Snowflake, or Redshift**.
- Orchestrate ingestion and dbt transformations using a tool such as **Airflow** instead of running scripts manually.
- Use **incremental dbt models** so large tables process only new or changed records instead of rebuilding everything.
- Add **CI/CD** so dbt models and tests run automatically before changes are deployed.
- Add automated data-quality monitoring for issues such as missing data, schema changes, abnormal KPI movements, and pipeline failures.
- Store credentials using a proper secrets-management service rather than local environment files.
- Publish Power BI through a managed workspace with scheduled refreshes and controlled access.
- Add experiment monitoring so SRM, conversion rates, and statistical results are automatically validated as experiments run.

I would keep the same basic layered architecture because the separation between raw data, staging, intermediate models, marts, and BI already provides a good foundation.

The main change would be moving from a **manually operated analytics workflow to an automated, monitored, and scalable production pipeline**.

Question 25: Based on your analysis, what business recommendations would you make?

Answer:

I would focus on four areas where the analysis showed the clearest opportunities.

1. Improve Customer Retention

Repeat customers were only about **3% of the customer base**, but they generated roughly **1.9x higher lifetime GMV proxy** than one-time customers.

So I would prioritize:

- post-purchase engagement,
- personalized recommendations,
- win-back campaigns,
- and targeted offers for promising or high-value customers.

2. Reduce Late Deliveries

Late deliveries showed roughly **2.7x higher low-rating risk** than on-time deliveries.

I would therefore:

- monitor high-risk sellers and states,
- identify recurring logistics bottlenecks,
- improve delivery estimates,
- and prioritize operational improvements in high-volume regions with elevated delay rates.

3. Optimize the Product Funnel

In the synthetic digital funnel, the largest drop-off occurred between:

Product View → Add to Cart

So I would investigate:

- product-page quality,
- pricing competitiveness,
- shipping visibility,
- trust signals,
- and CTA effectiveness.

This stage would be a strong candidate for further experimentation.

4. Use Experimentation Before Large Product Changes

The checkout experiment showed a statistically significant improvement of about **3.9 percentage points**.

So instead of relying only on observational analytics, I would use controlled A/B testing for major product or funnel changes whenever possible.

Overall, my recommendation would be to combine **retention improvement, delivery reliability, funnel optimization, and experimentation** rather than optimizing only one part of the customer journey.

Question 26: What advanced SQL concepts did you use in this project?

Answer:

I used SQL extensively across the dbt transformation layer rather than relying only on simple filtering and aggregation.

The main SQL concepts I used included:

- **CTEs** to break complex transformations into readable steps.
- **Joins** across orders, customers, payments, products, sellers, reviews, and order items.
- **GROUP BY and aggregations** for GMV, order counts, customer metrics, seller performance, and marketing KPIs.
- **Window functions** for customer-level scoring, ranking, and analytical calculations.
- **CASE WHEN logic** for business classifications such as customer segments, delivery status, and experiment decisions.
- **Date functions** for monthly trends, customer cohorts, delivery durations, and reliable analysis periods.
- **Conditional aggregation** for metrics such as late-delivery rates and funnel conversions.

- **Percentile-based logic** for RFM scoring.
- **Multi-step aggregation** to prevent double counting when tables had different grains.

For example, instead of directly joining payments and order items to orders, I first aggregated those tables to the required order grain and then joined them into the final fact model.

The main focus was not using advanced SQL just for complexity; it was using SQL to create **reliable, reusable business models that Power BI could consume directly**.

Question 27: How did you design the Power BI dashboard, and what was your approach to making it business-friendly?

Answer:

I designed the Power BI report around **business questions rather than database tables**.

Instead of putting every metric on one page, I divided the dashboard into six focused pages:

- **Executive Overview** — overall GMV, orders, customers, AOV, reviews, and delivery performance
- **Customer & Retention** — repeat customers, RFM segments, cohort retention, and LTV proxy
- **Product & Seller** — category performance, seller concentration, and freight economics
- **Operations & CX** — late deliveries, review scores, state-level risk, and seller performance
- **Funnel & Marketing** — conversion funnel, channel performance, ROAS, and cost per purchase
- **Experimentation** — control vs treatment performance, uplift, p-value, confidence interval, SRM, and business impact

I used a consistent visual hierarchy:

- KPI cards at the top,
- detailed analytical visuals below,
- slicers only where they added useful interactivity,
- consistent formatting across pages,
- and page navigation for easier movement through the report.

I also intentionally avoided putting too much transformation logic inside Power BI. Most calculations were already prepared through dbt marts, while DAX was mainly used for dynamic measures and presentation.

Another important design decision was transparency. The funnel, marketing, and experimentation pages clearly state that those datasets are synthetic.

The objective was to make the dashboard usable as a **decision-support tool**, not just a collection of charts.

Question 28: What are the main limitations of this project?

Answer:

The project is comprehensive, but there are a few important limitations that I would clearly acknowledge in an interview.

The main limitations are:

- The Olist dataset is **historical and public**, so it does not represent a live production business.

- The digital funnel, marketing, purchase-value, and A/B testing layers are **synthetic**, because the original dataset does not contain those fields.
- GMV is not the same as company revenue because commission or marketplace-fee information is not available.
- Customer LTV is only an **LTV proxy** based on observed lifetime GMV and does not include margin, acquisition cost, refunds, or future expected value.
- Marketing efficiency is measured as **cost per purchase**, not true CAC, because the synthetic data does not identify whether every purchaser is a new customer.
- Observational findings, such as the relationship between late delivery and poor ratings, show **association rather than causation**.
- The pipeline currently runs in a local environment and is not yet fully orchestrated or monitored like a production cloud system.

I see these limitations as important because a good analyst should communicate not only what the data shows, but also **what conclusions the data cannot support**.

If this were extended further, I would prioritize cloud deployment, automated orchestration, incremental processing, monitoring, and richer profitability data.

Question 29: What would you do if the A/B test result was not statistically significant?

Answer:

If the experiment were not statistically significant, I would **not automatically conclude that the treatment has no effect**.

First, I would check whether the experiment itself was valid by reviewing:

- sample ratio mismatch,
- randomization quality,
- tracking issues,
- experiment duration,
- sample size,
- and whether the planned minimum detectable effect was realistic.

Then I would look at the **confidence interval**, not just the p-value.

For example, a non-significant result could mean:

- there is genuinely little or no treatment effect,
- the sample size is too small,
- the observed effect is too uncertain,
- or the true effect is smaller than the experiment was powered to detect.

I would also avoid extending the experiment repeatedly just to obtain significance, because that increases the risk of false positives.

The business decision would therefore depend on the effect size, confidence interval, implementation cost, and experiment quality.

The key principle is:

“Not statistically significant” does not necessarily mean “no effect”; it means the available evidence is insufficient to confidently establish an effect.

Question 30: How did you validate that the numbers shown in Power BI were correct?

Answer:

I did not treat the Power BI output as the source of truth. The source of truth was the validated PostgreSQL/dbt analytical layer.

I used three levels of validation:

- **dbt tests** to validate model-level assumptions such as uniqueness, nulls, relationships, and business rules.
- **SQL reconciliation** to compare important mart outputs with their underlying fact tables.
- **Power BI cross-checks** to confirm that KPI cards and charts matched the corresponding dbt/PostgreSQL results.

For example, for metrics such as:

- GMV,
- delivered orders,
- repeat customers,
- late-delivery rate,
- conversion rate,
- ROAS,
- and experiment uplift,

I compared the dashboard output with the corresponding SQL or dbt mart result.

I also checked filter behavior carefully, especially the reliable analysis period and delivered-order filters, because incorrect filter context in Power BI can produce technically valid but misleading numbers.

So my validation approach was:

Raw data validation → dbt model testing → SQL reconciliation → Power BI verification.

This gave me confidence that the dashboard was presenting the same business logic that had already been validated upstream.

Question 31: How did you define GMV in this project, and why did you avoid calling it revenue?

Answer:

I defined **GMV, or Gross Merchandise Value**, as the total merchandise value of products sold through delivered orders.

In the project, the core calculation is based on the product value associated with completed transactions.

I intentionally used the term **GMV instead of revenue** because the Olist dataset does not provide enough information to calculate the marketplace's actual revenue.

True marketplace revenue would require additional information such as:

- seller commissions,
- platform fees,
- service charges,
- refunds,
- discounts funded by the platform,
- and other monetization components.

For example, if a customer buys a product worth R\$200, the marketplace may facilitate R\$200 of GMV but earn only a fraction of that amount as revenue.

So calling the full transaction value “revenue” would overstate the company's financial performance.

The principle I followed was:

Use GMV for transaction value unless the dataset contains the information required to calculate actual company revenue.

Question 32: Why did you use customer_unique_id instead of customer_id for customer-level analysis?

Answer:

In the Olist dataset, customer_id identifies a customer record associated with a specific order, while customer_unique_id is the persistent identifier that represents the same customer across multiple purchases.

For customer-level analysis such as:

- repeat purchase rate,
- RFM segmentation,
- cohort retention,
- customer lifetime GMV,
- and LTV proxy,

I needed a stable identifier across orders.

So I used:

customer_unique_id

as the customer key.

If I had used customer_id, the same real customer could appear as multiple different customers across different orders, which would:

- understate repeat purchases,

- inflate customer counts,
- distort retention,
- and make customer-level value metrics inaccurate.

So the principle was:

Use the identifier that represents the business entity consistently across transactions, not just the row-level operational identifier.

Question 33: How did you handle missing values and incomplete data in the project?

Answer:

I did not use a single blanket strategy for missing values. I handled them based on the **business meaning of each column**.

First, I profiled the source data to identify where nulls occurred and whether they represented:

- genuine missing information,
- fields that were not applicable,
- incomplete transactions,
- or source-quality issues.

For example, some products had missing metadata such as category or photo information. I retained those records rather than dropping the entire product because the transactional data was still useful.

For analytical metrics, I excluded rows only when the missing field was essential to that specific calculation.

For example:

- delivery analysis required valid delivery information,
- review analysis required available review scores,
- and reliable monthly trends excluded incomplete boundary periods.

I also documented known source-quality issues through dbt tests. One expected warning remained for missing `product_photos_qty`, rather than artificially filling those values just to make all tests pass.

My general approach was:

Understand why a value is missing first, then decide whether to retain, exclude, flag, or transform it based on the business use case.

I avoided arbitrary imputation because filling missing values without a defensible reason can introduce more bias than leaving them missing.

Question 34: How did you perform cohort retention analysis, and what did it tell you?

Answer:

I performed cohort analysis by grouping customers according to the **month of their first purchase**.

For each customer, I identified:

- first purchase month,
- subsequent purchase months,
- number of months since the first purchase,
- and whether the customer returned in each later month.

Then I aggregated this into a cohort-retention matrix where:

- **Rows** represent the customer's first-purchase month,
- **Columns** represent months since acquisition,
- **Values** represent the percentage of customers who returned.

For example:

- Month 0 represents the acquisition month and starts at 100%.
- Month 1 shows how many customers returned one month later.
- Month 2 shows how many returned two months later, and so on.

I also created a complete cohort grid so months with zero returning customers appeared explicitly instead of disappearing from the analysis.

The main insight was that retention dropped sharply after the first purchase, which was consistent with the overall repeat customer rate of only around **3%**.

This helped me conclude that customer retention is one of the biggest opportunities in the business.

The advantage of cohort analysis over a simple repeat-rate metric is that it shows **when customers return and how retention changes over time**, not just whether they ever purchased again.

Question 35: Can you explain the lineage of one KPI from raw data to the Power BI dashboard?

Answer:

A good example is **Total GMV**.

The flow was:

Raw order-item data → staging model → intermediate model → fact/mart layer → Power BI measure

First, raw order-item data was loaded into PostgreSQL.

In dbt, the staging model standardized the source fields and data types. Then intermediate models enriched and aggregated the item-level information while preserving the correct grain.

The cleaned transactional logic ultimately fed the analytical fact and mart models used by Power BI.

In Power BI, Total GMV was calculated only for:

- **delivered orders**
- within the **reliable analysis period from January 2017 to August 2018**

So the dashboard did not simply sum every transaction in the raw dataset.

This lineage was important because it made the KPI traceable:

source record → transformation → validated analytical model → dashboard metric

If a dashboard number looked incorrect, I could trace it backward through the dbt lineage and validate the calculation at each stage instead of debugging only inside Power BI.

Question 36: Why did you use Import mode in Power BI instead of DirectQuery?

Answer:

I used **Import mode** because the project dataset is moderate in size and the dashboard does not require real-time querying.

With Import mode, Power BI loads the analytical tables into its in-memory engine, which gives:

- faster visual interactions,
- better performance for DAX calculations,
- smoother filtering and slicing,
- and less dependency on PostgreSQL for every dashboard interaction.

Most of the heavy transformation work was already completed upstream using PostgreSQL and dbt, so Power BI mainly consumed clean fact tables and pre-aggregated marts.

DirectQuery would make more sense if:

- the dataset were too large to import,
- near-real-time data were required,
- or organizational rules required queries to remain in the source system.

For this project, Import mode provided the best balance of **performance, simplicity, and dashboard responsiveness**.

Question 37: How did you determine whether the A/B test had enough sample size?

Answer:

Before interpreting the experiment, I considered the required sample size using three main inputs:

1. **Significance level:** 5%
2. **Statistical power:** 80%
3. **Minimum Detectable Effect (MDE):** approximately a 3 percentage-point improvement in conversion

The baseline checkout-to-purchase conversion rate was around **62%**.

For detecting an increase of roughly:

62% → 65%

the required sample size was approximately:

4,000 users per group, or around **8,000 total users**.

The experiment actually contained:

- Control: **5,865 users**
- Treatment: **5,841 users**

So the available sample size was above the approximate requirement.

This was important because a statistically insignificant result with an underpowered experiment might simply mean that there was not enough data to detect a meaningful effect.

The principle I followed was:

Sample size should be planned from the expected baseline conversion, desired MDE, significance level, and statistical power—not chosen after seeing the result.

Question 38: Can you give an example where data validation changed your analysis or conclusion?

Answer:

Yes. One important example was the **delivery performance vs customer review analysis**.

Initially, I compared review timing and delivery timing directly using timestamps. During validation, I realized that this could misclassify some orders because the business question was whether the customer reviewed the order before or after the **delivery date**, not whether two timestamps matched at the exact time level.

So I corrected the logic to compare the relevant **calendar dates** and rebuilt the analysis.

After the correction, the reliable delivered-order results showed:

- On-time orders: average review score **~4.29**
- Late orders: average review score **~3.53**
- On-time low-rating rate: **~9.2%**
- Late low-rating rate: **~25.1%**

This produced a low-rating risk ratio of roughly **2.7×** for late deliveries.

The important lesson for me was that a query can run successfully and still produce the wrong business interpretation.

So I always validate:

data type → grain → business definition → calculation → final metric

before treating an analytical result as trustworthy.

Question 39: If one of the dashboard KPIs suddenly changed significantly, how would you investigate it?

Answer:

I would investigate the change systematically rather than immediately assuming it represents a real business trend.

First, I would check whether the change is caused by a **data or pipeline issue** by validating:

- source-data freshness,
- row counts,
- missing or duplicate records,
- dbt test results,
- and whether any transformation logic recently changed.

If the data pipeline is healthy, I would then break the KPI down by relevant dimensions.

For example, if GMV suddenly dropped, I would analyze it by:

- time period,
- customer state,
- product category,
- seller,
- order volume,
- and average order value.

Since:

$$\text{GMV} \approx \text{Order Volume} \times \text{Average Order Value}$$

I would first determine whether the decline came from fewer orders, lower order values, or both.

Similarly, for a conversion-rate decline, I would inspect the individual funnel stages to identify where the additional drop-off occurred.

Finally, I would compare the change with historical patterns and relevant operational factors before drawing a business conclusion.

My approach would therefore be:

Validate the data first → decompose the metric → identify the affected segment or funnel stage → investigate the business cause.

Question 40: How would you explain this project in 60-90 seconds to an interviewer?

Answer:

CommerceIQ is an end-to-end e-commerce analytics platform that I built using **Python**, **PostgreSQL**, **dbt**, **advanced SQL**, and **Power BI**.

I used the Olist e-commerce dataset as the transactional foundation and created a layered analytics pipeline:

Raw data → Staging → Intermediate models → Fact/Dimension models → Business marts → Power BI

The project covers several business areas including:

- customer segmentation and retention,
- product and seller performance,
- delivery and customer experience,
- funnel and marketing analytics,
- and A/B experimentation.

I also added synthetic clickstream, marketing, and experiment data because the original dataset did not contain those areas.

One of the strongest findings was that late deliveries had about **2.7× higher low-rating risk**, while repeat customers generated roughly **1.9× higher lifetime GMV proxy**.

I also designed a randomized checkout experiment where the treatment improved conversion by about **3.9 percentage points**, and I validated the result using SRM checks, a two-proportion z-test, p-value, and confidence interval.

Finally, I built a six-page Power BI dashboard to present the findings in a business-friendly way.

The main value of the project is that it demonstrates the full analytics lifecycle—from raw data and data modeling to statistical analysis and business decision-making.

Question 41: If you had more time, what would you improve or extend in this project?

Answer:

If I had more time, I would focus mainly on making the project more production-ready rather than adding many more analyses.

The main improvements would be:

- move PostgreSQL to a cloud warehouse such as **BigQuery, Snowflake, or Redshift**,
- automate the pipeline using **Airflow** or another orchestration tool,
- implement **incremental dbt models** for large tables,
- add **CI/CD** so dbt tests run automatically before deployment,
- add automated data-quality and anomaly monitoring,
- schedule Power BI refreshes through a managed workspace,
- and replace synthetic digital data with real clickstream, campaign, and experiment data if available.

I would also extend the customer side with more advanced modeling such as:

- churn prediction,
- customer propensity modeling,
- and personalized recommendation analysis.

However, I would prioritize **automation, monitoring, and scalability first**, because the analytical foundation is already complete.

The next step would therefore be to evolve the project from a strong analytical prototype into a **production-grade analytics platform**.

Question 42: How would this architecture handle a much larger dataset?

Answer:

The current architecture is logically scalable because the transformation layers are already separated, but for much larger data volumes I would change the execution environment and processing strategy.

I would mainly improve four areas:

- move from local PostgreSQL to a cloud data warehouse such as **BigQuery, Snowflake, or Redshift**,
- use **incremental dbt models** instead of rebuilding large tables from scratch,
- partition or cluster large fact tables by fields such as date,
- and orchestrate the pipeline so ingestion and transformation run automatically.

I would also avoid sending unnecessary raw-level data into Power BI.

Instead, I would keep Power BI connected mainly to:

- curated fact tables,
- dimensions,
- and pre-aggregated marts.

For very large BI workloads, I would also evaluate whether **Import mode, DirectQuery, or a composite model** is more appropriate depending on freshness and scale requirements.

The key point is that I would keep the same logical structure:

Raw → Staging → Intermediate → Facts/Dimensions → Business Marts → BI

but make the storage, processing, and refresh strategy more scalable.

Question 43: Why should an interviewer trust the analytical results from this project?

Answer:

I would not ask someone to trust the dashboard only because the visuals look correct. I built validation into multiple layers of the project.

My validation approach was:

Source profiling → dbt tests → reconciliation checks → business-rule validation → Power BI cross-checks

I used:

- uniqueness and not-null tests,

- relationship tests,
- accepted-value tests,
- custom dbt tests,
- reconciliation between marts and source models,
- grain validation to prevent double counting,
- and manual checks for important business definitions.

The final dbt build had:

480 passes, 1 expected warning, and 0 errors.

I also documented limitations instead of overstating results. For example:

- I use **GMV**, not revenue.
- I use **LTV proxy**, not true LTV.
- I use **cost per purchase**, not CAC.
- Observational relationships are not presented as causal.
- Synthetic funnel, marketing, and experiment data are clearly identified.

So the reliability of the project comes from both **technical validation and careful interpretation of what the data can and cannot support**.

This is the final high-priority question I would include in the core interview-preparation set.

Question 44: Why did you choose PostgreSQL for this project?

Answer:

I used PostgreSQL because it provided a proper relational database environment for building an analytics pipeline rather than analyzing CSV files directly in Python or Power BI.

It allowed me to:

- store raw and transformed data centrally,
- write advanced SQL transformations,
- create separate schemas for raw and analytical layers,
- integrate directly with dbt,
- enforce structured data types,
- and connect the final analytical models to Power BI.

For the scale of this project, PostgreSQL was sufficient and kept the setup manageable while still representing a realistic warehouse-style workflow.

If the same solution needed to handle much larger production workloads, I would consider migrating the analytical layer to a cloud warehouse such as **BigQuery, Snowflake, or Redshift**.

So PostgreSQL was a practical choice for demonstrating the complete analytics architecture without adding unnecessary cloud infrastructure at the portfolio stage.

Question 45: How did you decide which KPIs to include in the Executive Overview dashboard?

Answer:

I selected KPIs that together give a quick view of **business scale, customer activity, commercial value, and customer experience**.

The main KPIs were:

- **Reliable Delivered Orders** — transaction volume
- **Total GMV** — merchandise value generated
- **Average Order Value** — value per order
- **Active Customers** — customer participation
- **Average Review Score** — overall customer satisfaction
- **Late Delivery Rate** — operational service quality

I deliberately avoided adding too many KPIs because an executive page should highlight the most decision-relevant metrics rather than display every available measure.

I also made sure that the transactional KPIs used the same reliable analysis period and delivered-order definition so that the numbers were directly comparable.

The principle I followed was:

Each KPI should answer a distinct business question and together provide a balanced view of growth, customers, value, and operational health.

Question 46: How did you decide which dbt models should be views and which should be tables?

Answer:

I chose dbt materializations based on how frequently the model would be reused and how expensive it would be to recompute.

I generally kept:

- **Staging models as views**
- **Intermediate models as views**
- **Final marts as tables**

Staging and intermediate models mainly clean, standardize, or combine upstream data, so keeping them as views avoided creating unnecessary physical tables.

Final marts, on the other hand, were used repeatedly by Power BI and downstream analysis. Materializing them as tables improved query performance because the aggregation logic did not need to run every time the dashboard queried the database.

For example, marts such as:

- `agg_customer_segments`
- `agg_cohort_retention`
- `agg_category_performance`

- agg_marketing_roas
- agg_experiment_decision

were designed as reusable analytical outputs, so table materialization made more sense.

The principle I followed was:

Use lightweight views for transformation layers and persist frequently queried, computation-heavy analytical outputs as tables.

For a larger production system, I would also consider **incremental models** for high-volume fact tables so only new or changed data is processed during each run.

Absolutely — here is the **rapid-revision version** of the full CommerceIQ interview set. Each answer is intentionally kept to **1-3 lines** so you can revise the project quickly before an interview.

CommerceIQ — Rapid Revision Interview Q&A

Q1. Explain your CommerceIQ project end-to-end.

CommerceIQ is an end-to-end e-commerce analytics platform using Python, PostgreSQL, dbt, SQL, and Power BI. The flow is **Raw Data → Staging → Intermediate → Facts/Dimensions → Business Marts → Statistical Analysis → Power BI**.

Q2. Why did you use dbt?

dbt made the transformation layer modular, reusable, testable, and easier to maintain. I kept core business logic in dbt instead of duplicating it across Power BI reports.

Q3. Why did you use a star schema?

A star schema simplified BI analysis by separating facts from descriptive dimensions. I used fct_orders, fct_order_items, and dimensions such as customers, products, sellers, and date.

Q4. How did you ensure data quality?

I used source profiling, dbt uniqueness/not-null/relationship tests, custom business-rule tests, and reconciliation checks. Final dbt build: **480 passes, 1 expected warning, 0 errors**.

Q5. What were the main business insights?

Repeat customers generated about **1.9× higher LTV proxy**, late orders had about **2.7× higher low-rating risk**, and the synthetic A/B test produced about **+3.9 percentage-point conversion uplift**.

Q6. Why did you use Jan 2017-Aug 2018?

The first and last periods in Olist are incomplete. I used **Jan 2017-Aug 2018** as the reliable period to avoid distorted monthly trend comparisons.

Q7. How did you define repeat customers and retention?

I used customer_unique_id as the persistent customer key. Repeat customers had more than one delivered order, while cohort retention tracked customers returning in months after their first purchase.

Q8. How did you perform customer segmentation?

I used **RFM — Recency, Frequency, Monetary value** — with percentile-based scoring. Customers were grouped into segments such as Champions, Promising, Regular, Hibernating, and At-Risk High Value.

Q9. Why do you call it LTV proxy?

The dataset provides lifetime GMV but not margin, acquisition cost, servicing cost, or future value. Therefore, I call it an **LTV proxy**, not true financial customer lifetime value.

Q10. How did late delivery affect customer experience?

On-time orders had about **4.29 average rating and 9.2% low-rating rate**, while late orders had about **3.53 rating and 25.1% low-rating rate**. This is association, not causation.

Q11. What did seller concentration analysis show?

Top 1 seller contributed only about **1.7% of GMV** and top 10 about **13.3%**, indicating a relatively diversified seller base rather than dependence on a few sellers.

Q12. Why did you create synthetic data?

Olist lacks clickstream, marketing, and experiment data. I created clearly labeled synthetic datasets to demonstrate funnel analytics, ROAS, cost per purchase, and A/B testing.

Q13. How did you build the funnel?

I modeled **Session Start → Product View → Add to Cart → Checkout → Purchase** at session level. The biggest simulated bottleneck was **Product View → Add to Cart**.

Q14. How did you evaluate marketing performance?

I compared channels using CTR, CPC, CPM, cost per purchase, and ROAS. In the simulation, Email had the strongest efficiency, followed by Social and Paid Search.

Q15. How did you design the A/B experiment?

Visitors reaching checkout were randomly assigned to control or treatment at visitor_id level. The primary metric was checkout-to-purchase conversion.

Q16. Why did you perform an SRM check?

SRM checks whether the observed control/treatment allocation matches the intended split. The SRM p-value was about **0.8245**, so there was no evidence of allocation imbalance.

Q17. Which statistical test did you use?

I used a **two-proportion z-test** because I compared conversion rates between two independent groups with binary outcomes. The result had **$z \approx 4.36$ and $p \approx 0.000013$** .

Q18. Statistical significance vs business significance?

Statistical significance tells me whether the observed difference is unlikely due to chance; business significance asks whether the effect is large enough to matter commercially. I evaluated both.

Q19. How did you interpret the confidence interval?

The 95% CI for uplift was approximately **+2.15 to +5.65 percentage points**. Since the interval is entirely above zero, it supports a positive treatment effect.

Q20. Why connect Power BI to dbt marts instead of raw tables?

dbt marts already contained cleaned and validated business logic, which improved consistency, performance, and maintainability. Power BI mainly handled visualization and interactive measures.

Q21. What did you calculate in DAX vs dbt?

Reusable and heavy business logic stayed in dbt/SQL. DAX handled dynamic report measures such as GMV, AOV, active customers, late-delivery rate, and display-level experiment metrics.

Q22. How did you prevent double counting?

I explicitly defined the grain of every model. One-to-many tables such as payments and items were aggregated to the required grain before joining to the order-level fact table.

Q23. What was the biggest challenge?

Maintaining metric correctness across multiple grains and analytical layers was the hardest part. I solved it using clear grain definitions, intermediate models, and reconciliation tests.

Q24. How would you productionize this project?

I would move to a cloud warehouse, add orchestration, incremental dbt models, CI/CD, secrets management, automated monitoring, and scheduled Power BI refreshes.

Q25. What business recommendations would you make?

Focus on **customer retention, delivery reliability, funnel optimization, and experimentation**. These areas showed the strongest analytical opportunities in the project.

Q26. What advanced SQL concepts did you use?

I used CTEs, joins, aggregations, window functions, CASE logic, date functions, conditional aggregation, percentile-based scoring, and multi-step aggregation.

Q27. How did you design the Power BI dashboard?

I organized it around business questions, not tables. The six pages cover Executive, Customer, Product/Seller, Operations, Funnel/Marketing, and Experimentation.

Q28. What are the main limitations?

Olist is historical, digital layers are synthetic, GMV is not revenue, LTV is a proxy, cost per purchase is not CAC, and observational findings should not be interpreted causally.

Q29. What if the A/B test were not significant?

I would check experiment validity, sample size, power, confidence interval, and MDE before concluding anything. Non-significant does not automatically mean no effect.

Q30. How did you validate Power BI numbers?

I compared Power BI outputs against validated PostgreSQL/dbt marts and reconciliation queries. The dashboard was the presentation layer, not the source of truth.

Q31. Why GMV instead of revenue?

GMV represents merchandise value processed through the marketplace. Revenue would require commissions, fees, discounts, refunds, and other monetization information not present in the dataset.

Q32. Why customer_unique_id instead of customer_id?

customer_unique_id represents the same customer across purchases, while customer_id can change across orders. Using the wrong key would distort repeat-rate and retention analysis.

Q33. How did you handle missing values?

I handled nulls based on business meaning rather than applying one generic rule. I retained useful records, excluded only when necessary, and documented known data-quality issues.

Q34. How did cohort retention work?

Customers were grouped by first-purchase month, then tracked across later months. Rows represent acquisition cohorts, columns represent months since acquisition, and values represent retention rates.

Q35. Explain KPI lineage using GMV.

GMV flows from raw order-item data through staging, intermediate transformations, validated facts/marts, and finally Power BI. This makes the KPI traceable from source to dashboard.

Q36. Why Power BI Import mode?

The dataset was moderate in size and did not require real-time querying. Import mode provided faster visual performance and better dashboard responsiveness than DirectQuery.

Q37. How did you determine experiment sample size?

I considered baseline conversion, **5% significance level, 80% power, and ~3 percentage-point MDE**. Roughly 8K users were required, while the experiment had about **11.7K**.

Q38. Did validation ever change an analytical conclusion?

Yes. I corrected delivery/review timing logic after validating the relevant business dates, which materially improved the accuracy of the delivery-experience analysis.

Q39. How would you investigate a sudden KPI change?

First validate the data pipeline, then decompose the KPI by dimensions and components. For GMV, I would check whether the change came from order volume, AOV, category, seller, or geography.

Q40. Give a 60-90 second summary of the project.

CommerceIQ is an end-to-end e-commerce analytics platform combining SQL, dbt, PostgreSQL, Python, Power BI, customer analytics, marketing analytics, and experimentation. It demonstrates the full flow from raw data to validated business decisions.

Q41. What would you improve if you had more time?

I would focus on production readiness: cloud warehouse, orchestration, CI/CD, incremental models, monitoring, and replacing synthetic digital data with real event and marketing data.

Q42. How would the architecture scale?

The logical architecture can remain the same, but I would move storage to a cloud warehouse, use incremental models and partitioning, and keep Power BI focused on curated facts and marts.

Q43. Why should an interviewer trust your results?

Because I validated the project at multiple levels: source profiling, dbt tests, reconciliation checks, business-rule validation, and dashboard verification. I also clearly documented assumptions and limitations.

Q44. Why PostgreSQL?

PostgreSQL gave me a realistic relational analytics environment with strong SQL support, dbt integration, schema management, and direct Power BI connectivity without unnecessary cloud complexity.

Q45. How did you select Executive Overview KPIs?

I chose KPIs covering four dimensions: **volume, value, customers, and experience** — orders, GMV, AOV, active customers, review score, and late-delivery rate.

Q46. How did you choose dbt materializations?

I kept staging and intermediate models mostly as **views** and business marts as **tables**. This kept transformations lightweight while improving performance for frequently queried dashboard marts.

Last-Day Priority

If you have very little time, revise these most deeply:

Q1, Q2, Q3, Q4, Q5, Q7, Q10, Q15-Q19, Q22, Q23, Q25, Q28, Q30, Q40 and Q43.

Those are the questions most likely to reveal whether you genuinely understand and can defend the project.